

# Automation Template Selenium Java Cucumber

Manual de uso profesional

Template reutilizable de automatización Web UI

v1.1.0 en preparación

Hito 3 - Bloque 1: CI base y smoke con fixture local

Sin publicar

Baseline estable v1.0.2 - Stable / Validated

25 de septiembre de 2026

Java 21 (predeterminado) | Java 17 (compatible)

Selenium 4.48.0 | Cucumber 7.34.7 | JUnit 5.13.4 | Gradle 8.14.5

# Índice

|                                                     |    |
|-----------------------------------------------------|----|
| 1. Inicio rápido para primera vez .....             | 3  |
| 2. Arquitectura .....                               | 4  |
| 3. Stack, requisitos y VS Code .....                | 5  |
| 3.1 Configurar Java 17 paso a paso .....            | 6  |
| 4. Configuración y comandos .....                   | 7  |
| 5. Primera automatización .....                     | 8  |
| 6. Reutilización .....                              | 8  |
| 7. Contrato CI/CD, buenas prácticas y soporte ..... | 9  |
| 8. Glosario .....                                   | 10 |

## Propósito

El manual permite a una persona con conocimientos básicos o intermedios reutilizar el framework sin conocer previamente este repositorio. El ejemplo incluido es neutral y debe sustituirse por la aplicación bajo prueba.

## Información del documento

| Campo                        | Valor                                                |
|------------------------------|------------------------------------------------------|
| Autor / Creador del template | Israel Baz                                           |
| Rol                          | Test Automation / Prompt Engineering / AI Automation |
| Mantenimiento                | Israel Baz                                           |

## Estado e historial de release

| Versión | Fecha                    | Cambios principales                                                                                                                                                                          |
|---------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| v1.1.0  | En preparación           | Hito 3 - Bloque 1. Workflow base de GitHub Actions y smoke test con fixture HTML local. Validación local completada; sin commit ni publicación.                                              |
| v1.0.2  | 25 de septiembre de 2026 | Documentation-only Hotfix. Estado Stable / Validated; corrige inconsistencias documentales post-release sin cambios funcionales, dependencias, CI/CD, Docker, Grid, Healenium ni Playwright. |
| v1.0.1  | 25 de septiembre de 2026 | Hardening + CI/CD Readiness. Estado Stable / Validated / Published; tag v1.0.1 y GitHub Release publicados.                                                                                  |
| v1.0.0  | 17 de septiembre de 2026 | First Stable Release. Release histórica con Java 21 predeterminado, Java 17 configurable, documentación, ejemplo funcional y manual PDF.                                                     |

# 1. Inicio rápido para primera vez

Siga estos pasos sin modificar ningún archivo. Si un comando termina con BUILD SUCCESSFUL, se ejecutó correctamente. Si termina con BUILD FAILED, conserve el mensaje y consulte Troubleshooting antes de cambiar código.

## Paso 1 - Abra PowerShell y navegue a la raíz:

```
cd C:\ruta\automation-template-selenium-java-cucumber
```

## Paso 2 - Ejecute la prueba de ejemplo sin ventana visible:

```
.\gradlew.bat test -Dheadless=true
```

## Paso 3 - Abra el reporte de ejecución:

Abra build/reports/cucumber/cucumber.html. El escenario abre la fixture HTML local src/test/resources/fixtures/example\_page.html y valida el encabezado Example Domain. La fixture está versionada y no depende del contenido de un sitio externo; el escenario se reproduce en CI.

## Paso 4 - Ejecute con navegador visible:

Ejecute .\gradlew.bat test sin el parámetro headless.

## Cómo comprobar VS Code

Debe ver src, gradle, build.gradle y settings.gradle. Abra example\_domain.feature: debe aparecer coloreado como Gherkin. Abra ExampleDomainPage.java: Java no debe mostrar imports en rojo. Si VS Code solicita un JDK, seleccione Java 21 y espere que termine "Importing Gradle project".

## Lista antes de pedir ayuda

Estoy en la raíz del proyecto; java -version muestra Java 21; el navegador está instalado; la fixture local existe; la Feature termina en .feature y está bajo src/test/resources/features; cada frase Gherkin coincide con su Step; no guardé credenciales ni tokens en Git.

## 2. Arquitectura

El template estandariza la automatización Web UI: configuración externa, navegador, ejecución BDD, Page Object Model, evidencia y reportes. Es adecuado para proyectos que operan en Chrome o Edge. La configuración tiene prioridad JVM (-D), variable de entorno y archivo config.properties.

### Arquitectura general

| Capa             | Responsabilidad                                   |
|------------------|---------------------------------------------------|
| Feature          | Describe comportamiento en Gherkin.               |
| Step Definitions | Traduce intención a llamadas del framework.       |
| Page Objects     | Encapsula locators, esperas e interacciones.      |
| WebDriver        | Controla Chrome o Edge.                           |
| Hooks / reports  | Gestiona ciclo de vida, screenshots y resultados. |

### Flujo de ejecución

gradlew test -> Gradle compila -> JUnit Platform descubre el Runner -> Cucumber carga Features -> Hook Before crea WebDriver -> Steps llaman Pages -> Hook After captura evidencia y cierra -> Reporte HTML.

Regla de diseño: Steps no contienen selectores; Pages no contienen aserciones de escenario; Hooks no incluyen lógica de negocio.

### 3. Stack, requisitos y VS Code

| Tecnología           | Versión         | Uso                                 |
|----------------------|-----------------|-------------------------------------|
| Java                 | 21 (pred.) / 17 | Lenguaje y toolchain.               |
| Gradle Wrapper       | 8.14.5          | Build y dependencias reproducibles. |
| Selenium Java        | 4.48.0          | WebDriver.                          |
| Cucumber Java/Engine | 7.34.7          | BDD y Gherkin.                      |
| JUnit BOM            | 5.13.4          | JUnit Platform y Suite.             |

#### Prerrequisitos

Instale JDK 21 (recomendado) o JDK 17 (compatible), y Chrome o Edge. Use siempre el Gradle Wrapper incluido; en Windows, `.\gradlew.bat`. No requiere Gradle global. Valide con:

```
java -version
.\gradlew.bat --version
git --version
```

#### Apertura en Visual Studio Code

Abra PowerShell y ejecute:

```
cd C:\ruta\automation-template-selenium-java-cucumber
code .
```

Instale Extension Pack for Java, Gradle for Java, Cucumber (Gherkin) Full Support y GitLens. Espere que VS Code importe Gradle. Las Features están en `src/test/resources/features`; Pages en `src/main/java`; Steps, Hooks y Runner en `src/test/java`.

#### Selección de versión de Java

Java 21 es el predeterminado de v1.0.2. Java 17 es la única compatibilidad alternativa mediante la propiedad `Gradle javaVersion`. Java 18, 19, 20, 22 y cualquier otro valor distinto de 17 o 21 se rechazan explícitamente.

```
.\gradlew.bat clean test
.\gradlew.bat clean test -PjavaVersion=17
.\gradlew.bat clean test -PjavaVersion=21
.\gradlew.bat clean test -PjavaVersion=18 # rechazado
```

Gradle debe ejecutarse con JDK 17 o superior y la toolchain seleccionada debe estar instalada o disponible para Gradle.

## Sección 3.1 — Configurar Java 17 paso a paso

Use Java 17 solo cuando lo requiera su proyecto. Java 21 continúa siendo el predeterminado de v1.0.2.

**1. Instale un JDK 17 aprobado por su equipo. En estos ejemplos, la carpeta se representa como C:\ruta\jdk-17.**

**2. Abra PowerShell en la raíz del proyecto y configure la sesión actual. Esto no modifica permanentemente Windows:**

```
$env:JAVA_HOME = 'C:\ruta\jdk-17'
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
java -version
```

**3. Confirme que java -version indica Java 17. En VS Code, use Ctrl+Shift+P, ejecute Java: Configure Java Runtime, seleccione JDK 17 y espere la importación Gradle.**

**4. Ejecute las pruebas:**

```
.\gradlew.bat clean test -PjavaVersion=17
```

**5. Confirme BUILD SUCCESSFUL y abra build/reports/cucumber/cucumber.html. Para volver a Java 21, abra una consola nueva o cambie JAVA\_HOME a C:\ruta\jdk-21 y ejecute -PjavaVersion=21.**

No guarde rutas de JDK ni JAVA\_HOME dentro del repositorio. Para CI/CD, configure Java 17 o Java 21 mediante la imagen del agente o variables seguras de la plataforma.

## 4. Configuración y comandos

config.properties contiene los valores por defecto. Los valores pueden reemplazarse con -D o con variables BASE\_URL, BROWSER, HEADLESS, TIMEOUT\_SECONDS y SCREENSHOT\_ON\_FAILURE. javaVersion usa -P, no -D.

```
browser=CHROME
headless=false
baseUrl=https://example.com/
timeoutSeconds=15
screenshotOnFailure=true
```

| Objetivo           | Comando                                                                     |
|--------------------|-----------------------------------------------------------------------------|
| Limpiar            | .\gradlew.bat clean                                                         |
| Limpiar y ejecutar | .\gradlew.bat clean test                                                    |
| Headless           | .\gradlew.bat clean test -Dheadless=true                                    |
| Chrome             | .\gradlew.bat clean test -Dbrowser=CHROME                                   |
| Edge               | .\gradlew.bat clean test -Dbrowser=EDGE                                     |
| URL                | .\gradlew.bat clean test -DbaseUrl=https://su-aplicacion                    |
| Tags               | .\gradlew.bat clean test -Dcucumber.filter.tags=@example                    |
| Tags headless      | .\gradlew.bat clean test -Dheadless=true<br>-Dcucumber.filter.tags=@example |
| Alias Cucumber     | .\gradlew.bat cucumber                                                      |

El smoke test incluido usa la fixture HTML local versionada y no depende de un sitio externo. baseUrl sigue disponible para los Page Objects de aplicaciones reales, pero no modifica este escenario de ejemplo. Los reports se generan en build/reports/cucumber/cucumber.html, build/reports/tests/test y build/test-results/test; el log se guarda en build/logs/automation.log y las capturas de fallos en build/evidence/screenshots sólo si falla un escenario, screenshotOnFailure está activo y WebDriver permite capturar.

## 5. Crear la primera automatización

1. Cree src/test/resources/features/login.feature:

```
# language: en
Feature: Login
  Scenario: Successful login
    Given the user opens the login page
    When the user signs in with "username" and "password"
    Then the dashboard is displayed
```

2. Cree LoginPage.java en pages. Mantenga locators privados y métodos de intención, por ejemplo open(), login() e isDashboardVisible().

3. Cree LoginSteps.java en steps. Obtenga el driver mediante DriverManager, delegue a LoginPage y use aserciones JUnit.

4. Configure la URL con -DbaseUrl=... y ejecute .\gradlew.bat test -Dheadless=true.

5. Revise el reporte HTML. Use esperas explícitas WebDriverWait; nunca Thread.sleep.

## 6. Reutilización paso a paso

1. Copie o clone el template; conserve la base original sin cambios.

2. Actualice rootProject.name en settings.gradle y group en build.gradle.

3. Configure baseUrl con -DbaseUrl=https://su-aplicacion, sin secretos en Git.

4. Abra la URL manualmente y ejecute una prueba smoke en headless.

5. Cuando el smoke funcione, sustituya el ejemplo por sus Features, Pages y Steps.

6. Añada tags como @smoke y @regression para seleccionar subconjuntos.

7. Inicialice Git, revise .gitignore, cree una rama y abra Pull Request.

8. Conecte CI/CD con ejecución headless y publicación de reportes.

Criterio de salida: el equipo puede configurar URL, ejecutar una prueba y consultar el reporte sin editar componentes compartidos.

## 7. Contrato CI/CD, buenas prácticas y soporte

### Workflow base de GitHub Actions

El workflow `.github/workflows/ci.yml` ejecuta `./gradlew clean test -Dheadless=true` con Java 21 de Temurin y el Gradle Wrapper en pull requests y pushes hacia main. En Windows, use `./gradlew.bat clean test -Dheadless=true` para la validación local. Exit code 0 es éxito; cualquier otro código debe fallar el job. El contrato permite BROWSER, BASE\_URL, HEADLESS y filtros `cucumber.filter.tags`; recolecte `build/reports/cucumber/cucumber.html`, `build/reports/tests/test`, `build/test-results/test`, `build/logs/automation.log` y, sólo ante fallo, `build/evidence/screenshots`.

El workflow base usa `ubuntu-latest` y permisos mínimos de lectura. La publicación de artefactos, runners propios, Docker, Grid y secretos de CI/CD quedan fuera de este bloque.

### Troubleshooting de Gradle

`./gradlew.bat --stop` detiene Gradle Daemons ante problemas transitorios. Para diagnóstico de dependencias use `./gradlew.bat clean test --offline` o `./gradlew.bat clean test -PjavaVersion=21 --offline`; sólo usa caché y puede fallar si faltan dependencias. Si aparece `PKIX path building failed` o `unable to find valid certification path`, revise certificado Java, proxy, inspección SSL, red y daemon; ejecute `java -version`, `./gradlew.bat --version`, `./gradlew.bat --stop` y `./gradlew.bat clean test`. No deshabilite SSL ni ignore certificados.

### Regeneración del manual

`scripts/create_manual.py` requiere Python 3 y reportlab. Ejecute `python scripts/create_manual.py` desde la raíz. Genera `docs/Manual_Template_Automatizacion_Selenium_Java_Cucumber.pdf` y conserva una copia externa. `work/` es temporal, está ignorada por Git, no es parte del framework y puede eliminarse.

### Buenas prácticas

Un Page Object por pantalla/componente; nombres descriptivos; configuración externa; locators estables; datos aislados; screenshots y logs como evidencia; commits pequeños; ramas, Pull Requests y revisión de código; actualización deliberada de dependencias.

### Troubleshooting

| Síntoma                    | Solución inicial                                                    |
|----------------------------|---------------------------------------------------------------------|
| Java no reconocido         | Instale/configure JDK 21 o 17 y valide <code>java -version</code> . |
| Wrapper no ejecuta         | Ejecute desde la raíz y preserve <code>gradle/wrapper</code> .      |
| Browser no inicia          | Actualice Chrome/Edge y revise Selenium Manager.                    |
| Feature/Step no encontrado | Valide ruta <code>.feature</code> , tags, texto y paquete glue.     |
| CI falla y local no        | Compare variables, navegador, URL y artefactos.                     |

## 8. Glosario

Términos clave del template y de la automatización Web UI.

| Término              | Definición                                                              |
|----------------------|-------------------------------------------------------------------------|
| Automation Testing   | Ejecución de pruebas mediante software para validar una aplicación.     |
| Selenium             | Biblioteca que automatiza navegadores web.                              |
| WebDriver            | Interfaz que controla Chrome o Edge desde el código.                    |
| Cucumber             | Herramienta BDD que ejecuta escenarios escritos en Gherkin.             |
| BDD                  | Enfoque que describe el comportamiento esperado antes de implementarlo. |
| Gherkin              | Lenguaje legible para escribir Features, Scenarios y Steps.             |
| Feature              | Archivo o capacidad de negocio descrita en Gherkin.                     |
| Scenario             | Caso concreto que valida un comportamiento esperado.                    |
| Step Definition      | Código Java que implementa una frase de Gherkin.                        |
| Page Object Model    | Patrón que encapsula la UI y sus interacciones en clases Page.          |
| Hooks                | Código que prepara y limpia el estado antes o después de escenarios.    |
| Runner               | Clase que configura el descubrimiento y la ejecución de Cucumber.       |
| Gradle               | Herramienta de compilación, dependencias y ejecución de pruebas.        |
| Gradle Wrapper       | Scripts versionados que ejecutan la versión requerida de Gradle.        |
| JUnit Platform       | Plataforma que descubre y coordina la ejecución de pruebas.             |
| Headless             | Ejecución del navegador sin mostrar una ventana gráfica.                |
| CI/CD                | Integración y entrega continua mediante una pipeline automatizada.      |
| Pipeline             | Secuencia automatizada de compilación, pruebas y publicación.           |
| Git                  | Sistema de control de versiones distribuido.                            |
| GitHub Enterprise    | Plataforma corporativa para repositorios, revisiones y pipelines.       |
| Locator              | Regla que permite identificar un elemento en la interfaz.               |
| XPath                | Sintaxis para ubicar elementos a partir de la estructura del DOM.       |
| CSS Selector         | Sintaxis para ubicar elementos con atributos o clases CSS.              |
| Assertion            | Verificación que confirma el resultado esperado de una prueba.          |
| Test Data            | Datos controlados utilizados durante la ejecución de pruebas.           |
| Environment Variable | Valor externo al código que configura una ejecución.                    |
| Explicit Wait        | Espera controlada hasta que una condición de WebDriver se cumple.       |
| Screenshot           | Captura visual utilizada como evidencia, especialmente ante fallos.     |